Web Design and Development — Chapter 2 Review Answers
Comprehensive Solutions & Analysis for Chapter 2 Questions

A. Multiple Choice Questions

1. (Conceptual) In client-server architecture, which side starts an HTTP exchange?
a) The client ✓

b) The server
c) The DNS resolver
d) Whichever is less busy
Explanation: HTTP is a request-response protocol where communication is always initiated by the client (e.g., a web browser sending an HTTP GET request).
2. (Conceptual) HTTP is called stateless because:
a) it cannot transfer images
b) servers never store files
c) it has no status codes
d) each request is handled independently of earlier ones ✓

Explanation: Statelessness means the server does not automatically retain state or record user interactions across multiple HTTP requests without external mechanisms like cookies or sessions.
3. (Conceptual) What does a web server usually return when a URL's path names a folder?
a) A list of every file
b) A 500 error
c) The folder's default document, such as index.html ✓

d) The document root
Explanation: Web servers (like Apache or Nginx) are configured to serve default directory files—typically index.html or index.php—when a folder path is requested.
4. (Conceptual) What is the default port for HTTPS?
a) 21
b) 53
c) 80
d) 443 ✓

Explanation: Port 443 is the standard port for encrypted HTTPS traffic. (Port 80 is HTTP, 21 is FTP, 53 is DNS).
5. (Conceptual) Which server holds the actual DNS records for a domain?
a) The root server
b) The authoritative name server ✓

c) The recursive resolver

d) The web server

Explanation: The authoritative name server holds the official master DNS records (A, CNAME, MX, etc.) for a specific domain name.

6. (Conceptual) Why can a DNS change take hours to reach all users?

a) Resolvers cache answers until the TTL expires. ✓

b) Registrars approve changes manually.

c) IPv6 is slower than IPv4.

d) Browsers check DNS once a day.

Explanation: DNS records include a Time-To-Live (TTL) value. Recursive resolvers and local caches store the old IP address until that TTL duration expires.

7. (Syntax) In https://portal.example.edu:8443/grades?term=2#latest, what is the query?

a) 8443

b) /grades

c) term=2 ✓

d) latest

Explanation: The query string follows the question mark '?' in a URL (`term=2`). (`8443` is port, `/grades` is path, `#latest` is fragment).

8. (Syntax) How is a space written in a URL?

a) +20

b)

c) _

d) %20 ✓

Explanation: URL encoding (percent-encoding) replaces spaces with `%20` (or standard `+` in query parameters; `%20` is standard percent encoding).

9. (Code analysis) A page at /courses/web/intro.html links to ../index.html. Which path is requested?

a) /index.html

b) /courses/index.html ✓

c) /courses/web/index.html

d) /web/index.html

Explanation: Starting from `/courses/web/`, navigating up one directory (`..`) moves to `/courses/`, appending `index.html` gives `/courses/index.html`.

10. (Syntax) Which is a valid HTTP request line?

a) GET /news.html HTTP/1.1 ✓

b) HTTP/1.1 GET /news.html

c) /news.html GET HTTP/1.1

d) GET HTTP/1.1 /news.html

Explanation: An HTTP request line follows the strict order: `<METHOD> <REQUEST-URI> <HTTP-VERSION>`, such as `GET /news.html HTTP/1.1`.

11. (Output prediction) A browser asks for a page whose cached copy is still valid. Which status does the server return?

a) 200 OK

b) 301 Moved Permanently

c) 304 Not Modified ✓

d) 404 Not Found

Explanation: When conditional HTTP headers (like `If-Modified-Since`) indicate the content hasn't changed, the server returns `304 Not Modified` with no response body.

12. (Debugging) A page returns 500 Internal Server Error. Where should the developer look first?

a) The URL typed by the user

b) The browser's bookmarks

c) The DNS records

d) The server's program and error logs ✓

Explanation: A `500 Internal Server Error` indicates an unhandled server-side application exception or server configuration fault, which is recorded in the server logs.

B. True/False Questions

1. Over HTTP, a server can send a page that the browser did not request.

Answer: False

Explanation: HTTP is strictly a request-driven protocol; servers respond only to explicit client requests.

2. An IP address identifies a device, and a port identifies a program on it.

Answer: True

Explanation: IP addresses route network traffic to the physical/virtual host, while port numbers direct data to specific processes/services running on that host.

3. A domain name is read from left to right, most general part first.

Answer: False

Explanation: Domain names are hierarchical from right to left (TLD on the far right is most general, e.g., `.edu` -> `example` -> `portal`).

4. The fragment of a URL is sent to the server.

Answer: False

Explanation: URL fragments (the section following `#`) are processed purely client-side by the browser to scroll to anchor targets and are not transmitted in HTTP request lines.

5. A GET request normally has no body.

Answer: True

Explanation: GET requests are designed strictly to retrieve resources and carry parameters in the URL query string, omitting a request body.

6. POST data is encrypted even when sent over HTTP.

Answer: False

Explanation: HTTP transmits all data (including POST bodies) in plain unencrypted text. Encryption requires HTTPS (TLS/SSL).

7. A 4xx status code indicates a problem with the request.

Answer: True

Explanation: 4xx status codes (e.g., 400 Bad Request, 404 Not Found) denote client-side errors in request formatting, authorization, or resource targeting.

8. A valid certificate proves that a website is honest.

Answer: False

Explanation: A TLS certificate only verifies identity/ownership of the domain name and encrypts connection data—it does not guarantee that site operators are legitimate or honest.

C. Short Answer Questions

1. Explain client-server architecture and why the statelessness of HTTP matters.

Answer: • Client-Server Architecture: A distributed application structure that partitions tasks between resource/service providers (servers) and service requesters (clients, such as web browsers). • Why Statelessness Matters: Statelessness simplifies server design, allows web servers to process requests rapidly without holding persistent memory across sessions, and enables massive scalability/load balancing. However, it requires session mechanisms (like HTTP cookies, tokens, or local storage) to maintain user authentication and state.

2. Describe the steps of DNS resolution for a name that is not cached.

Answer: 1. Client Request: User enters domain; local OS sends query to the Recursive Resolver (ISP or 1.1.1.1). 2. Root Server Query: Resolver queries a DNS Root Server (`.`), which returns the address of the Top-Level Domain (TLD) server. 3. TLD Server Query: Resolver queries TLD server (e.g., `.edu`), which returns the Authoritative Name Server for the domain. 4. Authoritative Lookup: Resolver queries the domain's Authoritative Server, which returns the IP address. 5. Caching & Return: Resolver caches answer based on TTL and returns IP address to the browser.

3. Explain when to use absolute, relative, and root-relative URLs.

Answer: • Absolute URLs (`https://example.com/images/logo.png`): Use when referencing external resources on third-party domains. • Relative URLs (`../css/style.css`): Use when linking files within localized folder structures where portability across local development (e.g., Live Server) is needed. • Root-Relative URLs (`/assets/main.css`): Use for site-wide

global navigation or assets on large sites so links remain consistent regardless of current page depth.

4. Compare GET and POST, and state when to use each.

Answer: • GET Method: Appends data in URL query strings, has length limits, is cached/bookmarked, and must be idempotent (safe, read-only). Use for retrieving data/searching. • POST Method: Sends data inside the HTTP request body, has no length restrictions, is not cached, and changes server state. Use for form submissions, logins, file uploads, or database updates.

5. State the three guarantees of HTTPS and one thing the padlock does not prove.

Answer: • Three HTTPS Guarantees: 1. Confidentiality (Encryption prevents eavesdropping). 2. Integrity (Data cannot be modified/tampered with in transit). 3. Authentication (Verifies identity of server domain). • What Padlock Does NOT Prove: It does not prove that the website content is trustworthy, safe, or non-malicious (phishers can acquire valid TLS certificates for fake domains).

D. Essay Questions

1. Describe everything that happens, from DNS to rendered response, when a student submits a login form on a university portal over HTTPS, and identify where the process can fail.

Detailed Answer & Argument: Step-by-Step Lifecycle Process: 1. Form Submission & Client-Side Actions: Student clicks Submit. JavaScript validates inputs. The browser constructs an HTTP POST request carrying user credentials in the body. 2. DNS Lookup: Browser resolves portal domain via recursive DNS resolution hierarchy. 3. TLS Handshake: TCP connection on port 443 is established followed by TLS negotiation to establish session encryption keys. 4. HTTP POST Transmission: Secure POST request is transmitted across the internet to the application server. 5. Server Processing & Authentication: Back-end web application parses POST body, queries user database, hashes passwords, and verifies credentials. 6. Server Response & Cookie Header: Server responds with `302 Found` redirecting to dashboard and sets a `Set-Cookie: session_id=...` header. 7. Browser Rendering: Browser follows redirect, saves cookie, fetches dashboard resources, and renders DOM. Potential Failure Points: • DNS Failure: Domain unreachable or NXDOMAIN error. • TLS Failure: Expired or untrusted SSL certificate blocks connection. • Client Errors (4xx): Incorrect credentials return 401 Unauthorized or 403 Forbidden. • Server Errors (5xx): Database crash or uncaught server exception returns 500 Internal Server Error.

2. A small business owner asks why a brochure website with no forms needs HTTPS. Construct a reasoned argument.

Detailed Answer & Argument: Reasoned Argument for Universal HTTPS: 1. Data Integrity & Anti-Tampering: Without HTTPS, ISPs or public Wi-Fi routers can inject unwanted ads, malicious tracking code, or popups into plain HTTP sites. 2. Visitor Privacy: Plain HTTP leaks exactly which pages and products customers are browsing to eavesdroppers on local networks. 3. Browser Security Warnings: Modern browsers mark all HTTP sites as 'Not Secure', intimidating potential customers and damaging brand credibility. 4. Search Engine Optimization (SEO): Google and major search engines explicitly penalize plain HTTP websites in search rankings. 5. Modern Web Platform Features: PWA capabilities, geolocation, and HTTP/2 performance optimizations require HTTPS.

3. Explain how a developer uses status codes, headers, and developer tools to diagnose problems, using three realistic scenarios.

Detailed Answer & Argument: Scenario 1: Missing Image Asset (404 Error) • Symptom: Image fails to render on page. • DevTools Usage: Open Network Panel, filter by Images. Locate request highlighted in red showing status code `404 Not Found`. • Header/Diagnosis: Inspect request URL path to discover a typo in file capitalization or incorrect relative folder path. Scenario 2: CORS / Authorization Issue (403 Forbidden) • Symptom: API fetch call fails to fetch dynamic user data. • DevTools Usage: Open Console & Network Panel. Inspect failed API endpoint. • Header/Diagnosis: Check Response Headers for missing `Access-Control-Allow-Origin` or missing `Authorization: Bearer <token>` in Request Headers. Scenario 3: Server Script Exception (500 Internal Server Error) • Symptom: Form submission displays blank page or error banner. • DevTools Usage: Inspect Network POST request returning status code `500`. • Header/Diagnosis: View Response body payload in Network tab for backend stack traces or cross-reference backend log files.

E. Practical Programming Questions Solution Code

1. Basic — status-codes.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTTP Status Codes</title>
</head>
<body>
  <h1>HTTP Status Code Reference</h1>
  <h2>2xx Success</h2>
```

```
  <p>200 OK: Request succeeded. 201 Created: Resource created
successfully.</p>
  <h2>3xx Redirection</h2>
  <p>301 Moved Permanently: Resource URL changed. 304 Not
Modified: Cached copy valid.</p>
  <h2>4xx Client Errors</h2>
  <p>403 Forbidden: Access denied. 404 Not Found: Resource does
not exist.</p>
  <h2>5xx Server Errors</h2>
  <p>500 Internal Server Error: Server crashed. 503 Service
Unavailable: Server overloaded.</p>
</body>
</html>
```

2. Basic — links.html

```
<!DOCTYPE html>
<html lang="en">
<head><title>Link Types Test</title></head>
<body>
  <p><a href="https://developer.mozilla.org">Absolute External
Link</a></p>
  <p><a href="subfolder/page.html">Relative Link to
Subfolder</a></p>
  <p><a href="../index.html">Relative Parent Link (..)</a></p>
  <p><a href="/links.html">Root-Relative Link</a></p>
  <p><a href="?user=test&term=2">Query String Link</a></p>
  <p><a href="#bottom">Fragment Target Link</a></p>
  <div style="margin-top:1000px;" id="bottom">Fragment Target
Destination</div>
</body>
</html>
```

3. Intermediate — http-messages.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>HTTP Messages</title>
  <style>
```

```
    pre { background: #f4f4f4; padding: 10px; border-radius:
4px; }
    .request-line { color: #d97706; font-weight: bold; }
    .header { color: #0284c7; }
    .body { color: #16a34a; }
  </style>
</head>
<body>
  <h1>HTTP Request & Response Listings</h1>
  <pre>
<span class="request-line">GET /index.html HTTP/1.1</span>
<span class="header">Host: example.com</span>
<span class="header">Accept: text/html</span>
  </pre>
</body>
</html>
```

4. Intermediate — location-info.html

```
<!DOCTYPE html>
<html lang="en">
<head><title>URL Location Properties</title></head>
<body>
  <h1>URL Breakdown</h1>
  <div id="output"></div>
  <script>
    const props = ['href', 'protocol', 'host', 'pathname',
'search', 'hash'];
    const out = document.getElementById('output');
    props.forEach(p => {
      out.innerHTML += `<p><strong>${p}:</strong> ${location[p]}
</p>`;
    });
  </script>
</body>
</html>
```

6. Challenge — Request Builder App

```
<!DOCTYPE html>
<html lang="en">
<head><title>HTTP Request Builder</title></head>
<body>
  <h1>HTTP Request Builder</h1>
  <button onclick="build('GET', '/api/users')">GET
/api/users</button>
  <button onclick="build('POST', '/api/users')">POST
/api/users</button>
  <pre id="req-display"></pre>
  <script>
    function build(method, path) {
      let req = `${method} ${path} HTTP/1.1\nHost:
${location.host}\n`;
      if (method === 'POST') {
        req += `Content-Type: application/json\n\n{"name": "John
Doe"}`;
      }
      document.getElementById('req-display').textContent = req;
    }
  </script>
</body>
</html>
```